

MeetVideoCapture: техническое решение

Приложение захватывает уже отрисованное видео Google Meet или Zoom Desktop из top-level X11 окна, показывает сгор в Qt/QML preview и по кнопке пишет его в MKV/H.264.

Qt 6 / C++ / QML

X11 window capture

GStreamer

No browser / Zoom hacking

Что делаем

- Находим top-level окно Meet/Zoom/source app через X11.
- Захватываем pixels этого окна по `XID`.
- Обрезаем область meeting video внутри окна.
- Показываем только сгор и, при необходимости, пишем файл.

Чего принципиально не делаем

- Не перехватываем внутренний `WebRTC MediaStream`.
- Не вытаскиваем `VideoTrack` из браузера.
- Не используем tab capture API и Chrome extension.
- Не используем Zoom SDK для извлечения media stream.
- Не используем Selenium/CDP/injection/packet sniffing.

1. Основная идея

Google Meet в браузере и Zoom Desktop уже декодируют видео и рисуют его в своем top-level окне. Поэтому backend работает ниже уровня приложения: он берет готовую картинку окна из X11. Для Meet это не WebRTC stream, а для Zoom это не внутренний media stream Zoom.

```

Google Meet в браузере или Zoom Desktop
↓
source app декодирует и рисует видео
↓
X11 capture top-level source window by XID
↓
сгор области meeting video относительно source window
↓
Qt/QML preview latest frame
↓
optional MKV/H.264 recording
  
```

2. Захват окна

`X11WindowFinder` читает список top-level окон, получает title, className, XID, geometry, PID и source profile. Zoom meeting windows и browser meeting windows сортируются выше остальных, но выбрать можно любое top-level окно. После выбора controller передает XID и geometry в capture settings.

```

ximagesrc xid=<sourceWindowId> use-damage=false show-pointer=false ! video/x-raw,framerate=30/1 ! videocrop left=<l> top=<t> right=<r> bottom=<b> ! videoconvert ! video/x-raw,format=BGRA ! queue leaky=downstream max-size-buffers=1 ! appsink emit-signals=true sync=false max-buffers=1 drop=true
  
```

UI хранит сгор как `x/y/width/height`. GStreamer `videocrop` принимает `left/top/right/bottom`, поэтому controller пересчитывает значения от размеров выбранного окна:

```
left = cropX top = cropY right = sourceWidth - cropX - cropWidth bottom = sourceHeight - cropY - cropHeight
```

3. Preview без накопления задержки

Preview построен по latest-frame strategy: хранится только последний кадр. Если UI не успел отрисовать старый кадр, новый кадр его заменяет. Очереди preview нет, старые кадры не копят.

- `appsink drop=true`, `max-buffers=1`, `sync=false`.
- `queue leaky=downstream`, `max-size-buffers=1`.
- `VideoFrameProvider` хранит latest `QImage` и сжимает update-сигналы.
- `VideoPreviewItem` отрисовывает кадр через Qt Quick scene graph texture node.

4. Запись

Recording не блокирует UI и не тормозит preview. Во время записи capture frames подаются в `VideoRecorder`; когда запись выключена, этот feed отключен. Recorder имеет bounded queue и при перегрузе начинает dropping, а не увеличивает latency.

```
appsrc is-live=true format=time ! queue max-size-buffers=30 leaky=downstream ! videoconvert !
video/x-raw,format=I420 ! x264enc tune=zerolatency speed-preset=ultrafast key-int-max=30
bitrate=2500 ! h264parse ! matroskamux ! filesink location=<source-profile-recording-
YYYYMMDD-HHMMSS.mkv>
```

Формат по умолчанию - MKV/H.264. MKV выбран потому, что он надежнее для live recording: при аварийном завершении меньше риск получить полностью непригодный файл, чем с MP4.

5. Потоки

UI thread

QML, состояние controller, таймер записи, scene graph scheduling. Долгие X11/GStreamer команды сюда не попадают.

Window refresh worker

`X11WindowFinder` перечисляет top-level окна в отдельном `QThread` и возвращает готовый список в UI.

Capture command worker

Start/stop/restart/update crop идут последовательно в отдельном `QThread`, без блокировки QML.

Capture streaming path

GStreamer streaming threads, appsink callback, FPS stats, black/stale sampling, latest frame update.

Recorder worker

appsrc feeding, encoder/muxer/file sink, EOS/finalize без ожидания в UI thread.

Bounded data flow

Нет unbounded queues. Preview не ждет recorder. При перегрузе лучше drop, чем latency.

6. Fallback

Если XID capture дает черный или stale кадр, можно переключиться на `ScreenRegionFallback`. Тогда `ximagesrc` берет видимую область экрана по координатам crop. Это не tab capture и не extension: это захват pixels экрана.

Ограничение fallback: если поверх выбранной video region окажется другое окно, оно тоже попадет в capture.

7. Ограничения MVP

- Только Ubuntu X11. Wayland/PipeWire portal можно добавить отдельным backend позже.
- Свернутое или скрытое окно Meet/Zoom/source app может не рендериться.
- XID capture зависит от compositor/GPU/source app и иногда может быть black/stale.
- Crop задается численно относительно source window; визуальный overlay можно добавить позже.

8. Дальнейшее расширение на Windows и macOS

Linux/X11 backend должен оставаться первой реализацией общей архитектуры, а не отдельной Linux-only веткой. Для расширения на Windows и macOS платформенный захват нужно отделить от общей обработки кадров.

```

Platform capture backend
    ↓
common VideoFrame / frame abstraction
    ↓
crop / timestamp / format normalization
    ↓
common frame pipeline
    ↓
preview / analysis / recorder
  
```

Платформозависимыми остаются перечисление окон, выбор native source id, permission model, реальный capture API и работа с native pixel buffer или texture. Общими остаются frame abstraction, crop settings, timestamps, FPS/statistics, black/stale detection, bounded/latest-frame delivery, recorder interface, file naming и единая модель warnings/errors.

Linux

Текущий X11 путь: XID → GStreamer ximagesrc → crop → common frame. Для Wayland логичное расширение: PipeWire / xdg-desktop-portal → selected window или display stream → common frame.

Windows

Основной путь: HWND / selected window → Windows Graphics Capture → Direct3D texture frames → crop/convert → common frame. Не нужно строить backend на GDI screenshots или screenshot loop.

macOS

Основной путь: SCWindow или display → ScreenCaptureKit → CMSampleBuffer / CVPixelBuffer → crop/convert → common frame. Реализация частично будет Objective-C++.

Recorder

На первом этапе можно оставить общий recorder через GStreamer appsrc → x264enc → matroskamux → MKV/H.264, если GStreamer runtime поставляется вместе с приложением. Позже возможны native recorder backends: Linux → GStreamer, Windows → Media Foundation / hardware encoder, macOS → AVAssetWriter / VideoToolbox.

Ограничения Windows: может потребоваться user consent или picker, protected windows могут не захватываться, minimized window может не обновляться, а GPU/driver/compositor могут влиять на результат. В будущем желательно избегать лишнего GPU→CPU copy, если preview и recorder смогут работать с texture напрямую.

Ограничения macOS: нужен Screen Recording permission, пользователь может запретить доступ, protected content может не захватываться, minimized или hidden windows могут не обновляться.

Главное правило: не смешивать платформенный capture backend с общей логикой обработки и записи. Linux/X11, Windows и macOS должны быть разными реализациями одного capture interface. Тогда Ubuntu X11 MVP остается первой платформенной реализацией общей meeting/window capture architecture, а не Linux-only тупиком.

Итог: MeetVideoCapture не взламывает браузер, не трогает WebRTC и не извлекает внутренний media stream Zoom. Он захватывает то, что source app уже отрисовал в X11-окне, обрезает нужную область и передает ее в быстрый preview и recorder.